

DeepAgent MVP Implementation Guide

Table of Contents

1. [System Requirements](#)
2. [Installation & Setup](#)
3. [Configuration](#)
4. [Deployment Options](#)
5. [Integration Patterns](#)
6. [Monitoring & Logging](#)
7. [Troubleshooting](#)
8. [Production Considerations](#)

System Requirements

Minimum Requirements

- **Python:** 3.7 or higher
- **Memory:** 512MB RAM
- **Storage:** 100MB available space
- **CPU:** Single core sufficient for MVP

Recommended Requirements

- **Python:** 3.9+
- **Memory:** 2GB RAM
- **Storage:** 1GB available space
- **CPU:** Multi-core for concurrent processing

Dependencies

```
# Core dependencies
pip install jsonschema>=4.0.0

# Optional dependencies for production
pip install fastapi uvicorn # For API deployment
pip install redis           # For caching
pip install prometheus-client # For metrics
```

Installation & Setup

1. Environment Setup

```
# Create virtual environment
python -m venv deepagent_env
source deepagent_env/bin/activate # Linux/Mac
# deepagent_env\Scripts\activate # Windows

# Install dependencies
pip install jsonschema
```

2. Directory Structure Setup

```
# Clone or create the project structure
mkdir -p deepagent_mvp/{src,prompts,schemas,docs,tests,qa,integration,logs}

# Set permissions
chmod +x tests/run_test.sh
chmod +x src/deepagent_mvp.py
```

3. Configuration Validation

```
# Validate installation
python src/deepagent_mvp.py --help

# Run test suite
bash tests/run_test.sh

# Validate schemas
python -c "import jsonschema; print('JSON Schema validation available')"
```

Configuration

1. Schema Configuration

Edit `schemas/qa.json` to customize quality thresholds:

```
{
  "quality_metrics": {
    "phase1_metrics": {
      "literature_confidence_threshold": 0.6,
      "minimum_related_fields": 2,
      "minimum_key_concepts": 3
    },
    "phase2_metrics": {
      "feasibility_score_threshold": 0.5
    },
    "phase3_metrics": {
      "logical_consistency_threshold": 0.7,
      "max_acceptable_contradictions": 2
    }
  }
}
```

2. Prompt Customization

Modify prompt templates in `prompts/` directory:

- `phase1_academic_discovery.txt` - Academic analysis instructions
- `phase2_technical_feasibility.txt` - Technical assessment guidelines
- `phase3_contradiction_analysis.txt` - Contradiction detection criteria

3. Environment Variables

```
# Optional environment configuration
export DEEPAGENT_LOG_LEVEL=INFO
export DEEPAGENT_TIMEOUT=300
export DEEPAGENT_MAX_RETRIES=3
```

Deployment Options

1. Command Line Interface (CLI)

Basic usage for single insights:

```
# Process single insight
python src/deepagent_mvp.py \
  --input insight.json \
  --output result.json \
  --prompts-dir prompts \
  --schemas-dir schemas
```

2. Batch Processing

Process multiple insights:

```
# Create batch processing script
cat > batch_process.sh << 'EOF'
#!/bin/bash
for input_file in inputs/*.json; do
  output_file="outputs/$(basename "$input_file" .json)_result.json"
  python src/deepagent_mvp.py -i "$input_file" -o "$output_file"
done
EOF

chmod +x batch_process.sh
./batch_process.sh
```

3. API Service Deployment

Create a FastAPI wrapper:

```
# api_server.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import json
from src.deepagent_mvp import DeepAgentMVP

app = FastAPI(title="DeepAgent MVP API")
agent = DeepAgentMVP()

class InsightRequest(BaseModel):
    id: str
    content: str
    source: str
    timestamp: str
    metadata: dict = {}

@app.post("/analyze")
async def analyze_insight(request: InsightRequest):
    try:
        insight_data = request.dict()
        result = agent.process_insight(insight_data)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# Run with: uvicorn api_server:app --host 0.0.0.0 --port 8000
```

4. Docker Deployment

```
# Dockerfile
FROM python:3.9-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .
RUN chmod +x tests/run_test.sh

EXPOSE 8000
CMD ["python", "src/deepagent_mvp.py"]
```

```
# docker-compose.yml
version: '3.8'
services:
  deepagent-mvp:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./logs:/app/logs
      - ./data:/app/data
    environment:
      - DEEPAGENT_LOG_LEVEL=INFO
```

Integration Patterns

1. Pipeline Integration

```
# pipeline_integration.py
from src.deepagent_mvp import DeepAgentMVP
import json

class InsightPipeline:
    def __init__(self):
        self.agent = DeepAgentMVP()

    def process_stream(self, insight_stream):
        """Process streaming insights"""
        for insight in insight_stream:
            result = self.agent.process_insight(insight)
            yield result

    def batch_process(self, insights_batch):
        """Process batch of insights"""
        results = []
        for insight in insights_batch:
            result = self.agent.process_insight(insight)
            results.append(result)
        return results
```

2. Database Integration

```
# database_integration.py
import sqlite3
import json
from datetime import datetime

class InsightDatabase:
    def __init__(self, db_path="insights.db"):
        self.db_path = db_path
        self.init_database()

    def init_database(self):
        """Initialize database schema"""
        conn = sqlite3.connect(self.db_path)
        conn.execute('''
            CREATE TABLE IF NOT EXISTS insights (
                id TEXT PRIMARY KEY,
                content TEXT,
                source TEXT,
                timestamp TEXT,
                processing_result TEXT,
                verification_status TEXT,
                confidence_score REAL,
                created_at TEXT
            )
        ''')
        conn.commit()
        conn.close()

    def store_result(self, insight_id, result):
        """Store processing result"""
        conn = sqlite3.connect(self.db_path)
        conn.execute('''
            INSERT OR REPLACE INTO insights
            (id, processing_result, verification_status, confidence_score, created_at)
            VALUES (?, ?, ?, ?, ?)
        ''', (
            insight_id,
            json.dumps(result),
            result.get('summary', {}).get('verification_status'),
            result.get('summary', {}).get('confidence_score'),
            datetime.now().isoformat()
        ))
        conn.commit()
        conn.close()
```

3. Message Queue Integration

```
# queue_integration.py
import json
import redis
from src.deepagent_mvp import DeepAgentMVP

class QueueProcessor:
    def __init__(self, redis_host='localhost', redis_port=6379):
        self.redis_client = redis.Redis(host=redis_host, port=redis_port)
        self.agent = DeepAgentMVP()

    def process_queue(self, queue_name='insights'):
        """Process insights from Redis queue"""
        while True:
            # Blocking pop from queue
            item = self.redis_client.blpop(queue_name, timeout=30)
            if item:
                _, insight_data = item
                insight = json.loads(insight_data)

                # Process insight
                result = self.agent.process_insight(insight)

                # Store result
                result_key = f"result:{insight['id']}"
                self.redis_client.set(result_key, json.dumps(result))
```

Monitoring & Logging

1. Basic Logging Setup

```
# Add to src/deepagent_mvp.py
import logging
import os

# Configure logging
log_level = os.getenv('DEEPAGENT_LOG_LEVEL', 'INFO')
logging.basicConfig(
    level=getattr(logging, log_level),
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('logs/deepagent.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)
```

2. Performance Monitoring

```
# performance_monitor.py
import time
import json
from datetime import datetime

class PerformanceMonitor:
    def __init__(self):
        self.metrics = {
            'total_processed': 0,
            'processing_times': [],
            'error_count': 0,
            'verification_stats': {
                'verified': 0,
                'requires_review': 0,
                'significant_concerns': 0,
                'rejected': 0
            }
        }

    def record_processing(self, processing_time, result):
        """Record processing metrics"""
        self.metrics['total_processed'] += 1
        self.metrics['processing_times'].append(processing_time)

        status = result.get('summary', {}).get('verification_status', 'unknown')
        if status in self.metrics['verification_stats']:
            self.metrics['verification_stats'][status] += 1

    def get_stats(self):
        """Get performance statistics"""
        if self.metrics['processing_times']:
            avg_time = sum(self.metrics['processing_times']) / len(self.metrics['processing_times'])
            max_time = max(self.metrics['processing_times'])
            min_time = min(self.metrics['processing_times'])
        else:
            avg_time = max_time = min_time = 0

        return {
            'total_processed': self.metrics['total_processed'],
            'average_processing_time': avg_time,
            'max_processing_time': max_time,
            'min_processing_time': min_time,
            'error_count': self.metrics['error_count'],
            'verification_distribution': self.metrics['verification_stats']
        }
```

3. Health Check Endpoint

```
# health_check.py
from fastapi import FastAPI
import json
import os
from datetime import datetime

app = FastAPI()

@app.get("/health")
async def health_check():
    """System health check"""
    try:
        # Check file system access
        test_file = "logs/health_check.tmp"
        with open(test_file, 'w') as f:
            f.write("test")
        os.remove(test_file)

        # Check schema files
        schema_files = ['schemas/input.json', 'schemas/output.json', 'schemas/qa.json']
        for schema_file in schema_files:
            if not os.path.exists(schema_file):
                raise FileNotFoundError(f"Schema file missing: {schema_file}")

        return {
            "status": "healthy",
            "timestamp": datetime.now().isoformat(),
            "version": "1.0.0"
        }

    except Exception as e:
        return {
            "status": "unhealthy",
            "error": str(e),
            "timestamp": datetime.now().isoformat()
        }
```

Troubleshooting

Common Issues

1. Schema Validation Errors

```
# Check schema syntax
python -c "import json; json.load(open('schemas/input.json'))"

# Validate against sample
python -c "
import json, jsonschema
schema = json.load(open('schemas/input.json'))
sample = json.load(open('tests/sample_input.json'))
jsonschema.validate(sample, schema)
print('Validation successful')
"
```

2. Import Errors

```
# Check Python path
python -c "import sys; print('\n'.join(sys.path))"

# Check module imports
python -c "from src.deepagent_mvp import DeepAgentMVP; print('Import successful')"
```

3. Permission Issues

```
# Fix file permissions
chmod +x tests/run_test.sh
chmod +x src/deepagent_mvp.py
chmod -R 755 deepagent_mvp/
```

4. Memory Issues

```
# Monitor memory usage
python -c "
import psutil
import os
process = psutil.Process(os.getpid())
print(f'Memory usage: {process.memory_info().rss / 1024 / 1024:.2f} MB')
"
```

Debug Mode

Enable debug logging:

```
export DEEPAGENT_LOG_LEVEL=DEBUG
python src/deepagent_mvp.py -i tests/sample_input.json -o debug_output.json
```

Performance Profiling

```
# profile_performance.py
import cProfile
import pstats
from src.deepagent_mvp import DeepAgentMVP
import json

def profile_processing():
    agent = DeepAgentMVP()
    with open('tests/sample_input.json', 'r') as f:
        insight = json.load(f)

    # Profile the processing
    profiler = cProfile.Profile()
    profiler.enable()

    result = agent.process_insight(insight)

    profiler.disable()

    # Save profile results
    stats = pstats.Stats(profiler)
    stats.sort_stats('cumulative')
    stats.print_stats(20) # Top 20 functions

if __name__ == "__main__":
    profile_processing()
```

Production Considerations

1. Security

- **Input Validation:** Always validate inputs against schemas
- **Error Handling:** Don't expose internal errors to users
- **Access Control:** Implement authentication for API endpoints
- **Data Sanitization:** Clean input data before processing

2. Scalability

- **Horizontal Scaling:** Deploy multiple instances behind load balancer
- **Caching:** Cache frequently accessed data and results
- **Database:** Use proper database for persistent storage
- **Queue Management:** Implement proper queue management for high throughput

3. Reliability

- **Error Recovery:** Implement retry mechanisms with exponential backoff
- **Circuit Breakers:** Prevent cascade failures
- **Health Monitoring:** Continuous health checks and alerting
- **Backup Strategy:** Regular backups of configuration and data

4. Performance Optimization

- **Async Processing:** Use async/await for I/O operations
- **Connection Pooling:** Reuse database connections
- **Memory Management:** Monitor and optimize memory usage

- **Profiling:** Regular performance profiling and optimization

5. Maintenance

- **Version Control:** Proper versioning of schemas and prompts
- **Configuration Management:** Centralized configuration management
- **Documentation:** Keep documentation updated
- **Testing:** Comprehensive test suite with CI/CD integration

Next Steps

1. **Production Deployment:** Follow deployment patterns for your environment
2. **Falcon Integration:** Implement Falcon handoff procedures (see [FALCON_HANDOFF.md](#) (FALCON_HANDOFF.md))
3. **Custom Extensions:** Develop domain-specific analysis modules
4. **Monitoring Setup:** Implement comprehensive monitoring and alerting
5. **Performance Tuning:** Optimize for your specific use case and load patterns

For additional support, refer to the main [README.md](#) (README.md) and [FALCON_HANDOFF.md](#) (FALCON_HANDOFF.md) documentation.